

Computer Architecture – Homework #4

Introduction to GPU

Karel Barbera Morales & Esteban Mocskos

Objective

In the previous homework, you solved the 1D heat equation on a single CPU core and then used MPI to solve it across multiple nodes. In this assignment, you will port the 2D version of the problem to a GPU using `CUDA`.

We have to move to the 2D heat equation because it fits in the programming model of the GPU. The assignment has three parts:

- **Part A:** Naïve CUDA kernel: one thread per grid point, global memory only.
- **Part B:** Optimised CUDA kernel: tiled shared-memory with halo loading.

Because you are working with the same physical problem and boundary conditions as previous assignment but solving the 2D problem, you'll need to extend the sequential CPU implementation to 2D in order to establish a baseline for comparison.

Governing Equation

The 2D transient heat equation is:

$$\frac{\partial T}{\partial t} = \alpha \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right) \quad (1)$$

where $T(x, y, t)$ is temperature and α is the thermal diffusivity.

Explicit Finite Difference Discretisation

Using a forward-Euler time step on a uniform $N \times N$ grid with spacing $\Delta x = \Delta y = h$, the update rule is:

$$T_{i,j}^{n+1} = T_{i,j}^n + \alpha \frac{\Delta t}{h^2} (T_{i+1,j}^n + T_{i-1,j}^n + T_{i,j+1}^n + T_{i,j-1}^n - 4T_{i,j}^n) \quad (2)$$

for $i, j \in [1, N - 2]$ (interior points). The stability condition (CFL criterion) requires:

$$\alpha \frac{\Delta t}{h^2} \leq \frac{1}{4} \quad (3)$$

This update is a **5-point stencil**: each interior grid point is independent of all others in the same timestep. This property makes the computation very well-suited for GPU execution.

Arithmetic Intensity – Pre-lab Question

△ Note

Answer this question **before writing any code**. Record your answer in the report.

For a single point update using equation (2), count:

- **Floating-point operations:** additions, subtractions, multiplications.
- **Bytes transferred from/to memory:** how many distinct `float` (4-byte) values are read and written, assuming no cache reuse between neighbouring threads?

Compute the arithmetic intensity $I = \text{FLOPs/Bytes}$ and compare it to the ridge point of your GPU:

$$I_{\text{ridge}} = \frac{\text{Peak FP32 GFLOP/s}}{\text{Peak Memory Bandwidth (GB/s)}}$$

Is the stencil compute-bound or memory-bound on your GPU?

Environment and Getting Started

The following command queries the system to get the capabilities of all the available GPUs:

Listing 1: Build commands

```
# Check compute capability of the GPU
nvidia-smi --query-gpu=name,compute_cap --format=csv,noheader
```

You can check the list of capabilities in: <https://developer.nvidia.com/cuda/gpus>

The following code shows you how to enable the specific optimizations based on the architecture of the available GPU. Note that if you compile for a more advanced compute capability, this binary may not run on GPUs with older compute capabilities.

Listing 2: Build commands

```
# Replace sm_XX with your GPU's compute capability
# e.g. sm_86 (Ampere 8.6), sm_89 (Ada Lovelace 8.9), sm_90 (Hopper 9.0)
nvcc -O3 -arch=sm_XX -o heat_cuda heat_cuda.cu
```

Part 0: Sequential 2D Solver

Extend the previous 1D CPU implementation to solve the 2D heat equation.

- Validate the numerical solution.
- Measure the baseline execution time.

This implementation will serve as the reference against which GPU speedups are measured and GPU results are validated.

Deliverables

Part 0

- `heat_serial.c` (or C++, if you prefer).
- Plots showing the stationary solution and several intermediate time steps.
- Baseline execution times for the problem sizes used later in the assignment.

Part A: Naïve CUDA Kernel

Implement a CUDA kernel where **one thread updates one grid point**. Use a 2D grid of 2D thread blocks so that threads map naturally onto the (i, j) indices of the array.

Listing 3: Skeleton: naïve global-memory kernel (`heat_cuda.cu`)

```
1 // Each thread handles one (i, j) interior point.
2 // T_old and T_new are row-major flat arrays of size N*N.
3 __global__ void heat_naive(float* __restrict__ T_new, const float*
  __restrict__ T_old, int N, float r) {
4     int j = blockIdx.x * blockDim.x + threadIdx.x; // column
5     int i = blockIdx.y * blockDim.y + threadIdx.y; // row
6
7     if (i > 0 && i < N-1 && j > 0 && j < N-1) {
8         // TODO: implement equation (2)
9         // Access pattern: T_old[i*N + j], T_old[(i+1)*N + j], etc.
10    }
11 }
```

The `__restrict__` keyword in CUDA is a type qualifier used on pointers to inform the compiler that pointer aliasing will not occur. By applying it, you inform to the compiler that for the lifetime of that pointer, the underlying data it points to will not be accessed or modified by any other pointer in the same scope.

Your host code must:

1. Allocate device arrays with `cudaMalloc`.
2. Copy initial conditions and boundary values to the device.
3. Launch the kernel for `num_steps` iterations. After each step, **swap** the `T_old` and `T_new` device pointers. Do *not* copy between device and host each step.
4. Copy the final grid back to the host and store it following a format similar to that used in the previous homework.

The serial code terminates when $\max_{i,j} |T_{i,j}^{n+1} - T_{i,j}^n| < \varepsilon$. Implement the same check on the GPU using a **parallel reduction**:

1. Add a second kernel (or extend the first) that computes the block-local maximum difference into a shared-memory accumulator.
2. Write one value per block to a device array `d_block_max`.

3. The host reads `d_block_max` every `check_interval` steps (suggested: every 50 steps) and takes the global max with `std::max` over the array.

Hint

You may use `thrust::reduce` or `cub::DeviceReduce::Max` instead of writing the reduction kernel by hand. State clearly in your report which approach you used and why.

△ Note

Calling `cudaMemcpy` every time step defeats the purpose of GPU offloading. Check convergence only every `check_interval` steps.

Block Size Exploration

Run the naïve kernel on a grid of $N = 1024$ for 2000 steps with block sizes (8×8) , (16×16) , and (32×32) . Record wall-clock time and fill in the table below. If you have access to multiple GPUs, generate a table for each device and include its specifications

Block size	Threads/block	Time (ms)	Throughput (Gpt/s)
8×8	64		
16×16	256		
32×32	1024		

Answer the following questions:

1. Why might smaller or larger blocks underperform?
2. What is the maximum number of threads per block on your GPU? (Check with `nvidia-smi` or the CUDA device query.) Does the 32×32 configuration respect this limit?
3. Using the best block size from your table, compute the **achieved memory bandwidth** (GB/s):

$BW_{\text{achieved}} = \frac{(N - 2)^2 \times \text{bytes_per_point}}{\text{time_per_step}}$ where `bytes_per_point` is the total bytes read and written per update (use your answer from Section).

Deliverables

Part A

- `heat_cuda.cu` compiling and validated against `heat_serial`
- Completed block-size table for $N = 1024$
- Answers to the questions.
- Speedup of the best naïve configuration over the serial solution at $N \in \{512, 1024, 2048\}$.

Part B: Optimised Shared-Memory Kernel

Tiled Kernel with Halo Loading

Improve the naïve kernel by loading each block's tile and its surrounding **1-cell halo** border into shared memory before computing the stencil. This reduces global memory traffic because each interior value is loaded once from DRAM and reused by up to four neighbours within the same block.

Listing 4: Skeleton: shared-memory tiled kernel (`heat_cuda_tile.cu`)

```
1 #define TILE_W 16
2 #define TILE_H 16
3
4 // Shared tile dimensions include the 1-cell halo on all sides.
5 // Interior tile: TILE_H x TILE_W
6 // Shared array: (TILE_H+2) x (TILE_W+2)
7
8 __global__ void heat_shared(float* __restrict__ T_new, const float*
  __restrict__ T_old, int N, float r) {
9     __shared__ float s[TILE_H + 2][TILE_W + 2];
10
11     // Global indices of this thread's "owned" interior cell
12     int j = blockIdx.x * TILE_W + threadIdx.x;
13     int i = blockIdx.y * TILE_H + threadIdx.y;
14
15     // Shared-memory indices (offset by 1 for the halo)
16     int lj = threadIdx.x + 1;
17     int li = threadIdx.y + 1;
18
19     // -- Step 1: Load interior cells
20     if (i < N && j < N)
21         s[li][lj] = T_old[i * N + j];
22
23     // -- Step 2: Load halo cells
24     // Left halo: threads in column 0 load their left neighbour
25     if (threadIdx.x == 0 && j > 0)
26         s[li][0] = T_old[i * N + (j - 1)];
27     // TODO: right, top, bottom halos (analogous pattern)
28
29     __syncthreads();
30
31     // -- Step 3: Compute stencil from shared memory
32     if (i > 0 && i < N-1 && j > 0 && j < N-1) {
33         // TODO: implement equation (2) using s[li][lj] etc.
34         // Do NOT read from T_old here -- use s[][] only.
35     }
36 }
```

Halo loading requirements:

- **Left/right halos:** threads at the leftmost/rightmost column of the block load the adjacent global cell (or the boundary value if at the domain edge).

- **Top/bottom halos:** threads at the top/bottom row of the block load the adjacent global cell above/below.
- **Corners:** handled by the corner threads; assign consistently to avoid double-loading.
- All threads must call `__syncthreads()` *before* the compute phase, even those outside the active domain.

Hint

The halo loading here is the GPU analog of MPI halo exchange. In the MPI version, each rank sent boundary elements to neighbours over the network. Here, each thread block loads boundary values from global memory into shared memory before computation. Compare the two mechanisms (latency, bandwidth, programming effort) in your report.

Tile Size Exploration

Run the shared-memory kernel on $N = 1024$ for 2000 steps with tile sizes 8×8 , 16×16 , and 32×32 . Compare with the best naïve result from Part A.

Tile size	Time (ms)	Throughput (Gpt/s)	Speedup vs naïve	Shared mem / block (KB)
8×8				
16×16				
32×32				

The shared memory per block (in bytes) for a tile of size $T_H \times T_W$ is:

$$\text{Shared mem} = (T_H + 2)(T_W + 2) \times 4 \text{ bytes}$$

Answer the following questions:

1. How much shared memory does each SM on your GPU provide? Is any of your tile sizes limited by shared memory capacity?
2. Explain why the shared-memory kernel is faster than the naïve kernel for large N , but the advantage may shrink for small N .
3. Run both kernels at $N \in \{256, 512, 1024, 2048, 4096\}$ and plot throughput (grid-points/second) vs N for both on the same axes. At what grid size does each kernel peak, and why?

Deliverables

Part B

- `heat_cuda_tile.cu` using shared-memory, validated against `heat_serial`
- Completed tile-size table for $N = 1024$
- Throughput vs N plot (both kernels, $N = 256$ to 4096)
- Answers to questions

Submission Instructions

Submit a compressed file named `group_<group_letter>.zip` where `group name` is your assigned group letter.

The submission must contain:

- All source files (including the sequential 2D heat equation solver).
- Specifications of the platforms used to execute the codes.
- All SLURM scripts used for execution on the cluster.
- A PDF report including:
 - Answers to all analysis questions.
 - Performance measurements.
 - Scalability plots.
 - Discussion of results.
 - Description of encountered problems and solutions.
- Any scripts used for plotting or post-processing.